

SOFTWARE REQUIREMENTS SPECIFICATION

SimuLearn LMS Platform

Simulation-Based Learning Management System

Version 1.0

Date: June 2026

Classification: Internal / Development Team

Status: Draft

Table of Contents

Table of Contents	2
1. Introduction	5
1.1 Purpose	5
1.2 Scope	5
1.3 Intended Users	5
1.4 Definitions and Abbreviations	6
1.5 Product Vision	6
2. Overall Description	8
2.1 Product Perspective	8
2.2 Product Functions — High-Level Summary	8
2.3 Operating Environment	8
2.4 Design Constraints	9
2.5 Assumptions and Dependencies	9
3. User Roles and Responsibilities	10
4. Functional Requirements	11
4.1 Authentication and Authorization	11
4.2 User Management	11
4.3 Role and Permission Management	12
4.4 Institution and Department Management	12
4.5 Course Management	13
4.6 Module and Lesson Management	14
4.7 Virtual Simulation Catalog	14
4.8 Simulation Player and Launcher	15
4.9 Simulation Session Tracking	15
4.10 Assignments	16
4.11 Quizzes and Exams	16
4.12 Gradebook	17
4.13 Rubrics	18
4.14 Learning Outcomes and Competencies	18
4.15 Progress Tracking	18
4.16 Calendar	19
4.17 Announcements	19
4.18 Discussions	19
4.19 Messaging and Notifications	20
4.20 Certificates	20
4.21 Admin Dashboard	20
4.22 Reports and Analytics	21

4.23 Content Management	21
4.24 API and Integrations	22
4.25 Audit Logs and System Settings	22
5. Non-Functional Requirements	24
6. Use Cases	25
UC-01: Student Registration and Login.....	25
UC-02: Student Enrolling in a Course	25
UC-03: Student Starting a Virtual Simulation	26
UC-04: Student Completing a Simulation-Based Lab.....	27
UC-05: Student Submitting an Assignment	27
UC-06: Student Taking a Quiz	28
UC-07: Instructor Creating a Course.....	29
UC-08: Instructor Grading Student Work	29
UC-09: Admin Creating Users and Roles.....	30
UC-10: Simulation Developer Integrating a New Simulation	31
7. User Sequence Flows	32
7.1 Login Flow	32
7.2 Course Enrollment Flow.....	32
7.3 Simulation Launch Flow	32
7.4 Simulation Completion and Score Saving Flow.....	33
7.5 Assignment Submission Flow	33
7.6 Quiz Attempt Flow	34
7.7 Instructor Course Creation Flow.....	34
7.8 Admin User Management Flow	34
8. System Architecture.....	36
8.1 Architecture Overview	36
9. Database Design	38
users.....	38
institutions.....	38
roles & permissions	38
courses.....	39
modules & lessons.....	39
simulations.....	40
simulation_sessions.....	40
assignments & submissions.....	41
quizzes & questions.....	42
grades	42
enrollments	43
certificates	43

- announcements, discussions, messages 43
- audit_logs 44
- 10. API Requirements 45
 - 10.1 Auth 45
 - 10.2 Users 45
 - 10.3 Courses 45
 - 10.4 Modules & Lessons 46
 - 10.5 Simulations 46
 - 10.6 Assignments & Quizzes 46
 - 10.7 Grades & Analytics 47
 - 10.8 Notifications 47
- 11. UI Pages and Dashboards 48
- 12. Role-Based Permissions Matrix 50
- 13. Simulation Learning Features 52
 - 13.1 Simulation Metadata Schema 52
 - 13.2 Experiment Step Flow 52
 - 13.3 Auto-Grading Logic 52
 - 13.4 Competency Proficiency Tracking 53
 - 13.5 Feedback Engine 53
- 14. Reports and Analytics 54
- 15. Development Roadmap and Deliverables 56
 - 15.1 Recommended MVP Features (Phase 1 — 6 months) 56
 - 15.2 Phase 2 — Months 7–12 (Advanced Features) 56
 - 15.3 Phase 3 — Months 13–18 (Scale & Intelligence) 57
 - 15.4 Final Deliverables Summary 57

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for SimuLearn — a web-based Learning Management System (LMS) that integrates simulation-based virtual laboratory learning with comprehensive course management capabilities.

This document serves as the primary reference for product owners, UI/UX designers, frontend and backend developers, QA engineers, database architects, and DevOps teams throughout the design, development, and testing lifecycle.

1.2 Scope

SimuLearn is a full-featured, cloud-based LMS that combines:

- Labster-style virtual simulation laboratories with interactive experiment flows, real-time feedback, auto-grading, and competency tracking.
- Canvas LMS-style learning management with courses, modules, lessons, assignments, quizzes, gradebooks, calendars, discussions, and messaging.
- Institutional administration with multi-tenant support, domain/department management, role-based access control, and analytics dashboards.

The platform targets higher education institutions, vocational training centers, and corporate learning departments that require both theoretical course delivery and hands-on virtual experimentation.

1.3 Intended Users

User Role	Description
Students	Primary learners accessing courses, simulations, and assessments.
Instructors	Faculty who create, deliver, and assess course content.
Teaching Assistants	Support grading and student interaction under instructor supervision.
Content Creators	Specialists who produce and curate learning materials.
Simulation Developers	Technical authors who build and integrate virtual lab simulations.
Department Managers	Domain/department heads who oversee curriculum and enrollment.

Institution Admins	IT or academic administrators managing a single institution's platform.
Super Admins	Platform-level administrators managing multiple institutions and system settings.
Guest / Demo Users	Unauthenticated visitors previewing public demo simulations.

1.4 Definitions and Abbreviations

Term	Definition
LMS	Learning Management System — a software platform for delivering, tracking, and managing educational content.
SRS	Software Requirements Specification — this document.
Simulation	An interactive, browser-rendered virtual experiment that models real-world phenomena.
Module	A structured grouping of lessons, assignments, and simulations within a course.
Enrollment	The act of a student registering to access a course.
SCORM	Sharable Content Object Reference Model — a standard for packaging and tracking e-learning content.
xAPI	Experience API — a specification for tracking learning experiences.
LTI	Learning Tools Interoperability — a standard for integrating third-party tools into an LMS.
Gradebook	A record of student scores across all graded activities in a course.
Rubric	A scoring guide used to evaluate student work against defined criteria.
Competency	A measurable skill or knowledge outcome mapped to learning activities.
API	Application Programming Interface — a set of HTTP endpoints for system integration.
JWT	JSON Web Token — a standard for secure token-based authentication.
SSO	Single Sign-On — allows users to authenticate once across multiple systems.
RBAC	Role-Based Access Control — restricting system access based on assigned user roles.
CDN	Content Delivery Network — distributed servers for fast static asset delivery.
SLA	Service Level Agreement — contractual performance and availability commitments.

1.5 Product Vision

Vision Statement

SimuLearn empowers educators and learners by unifying the depth of simulation-based scientific exploration with the organizational power of a full-featured LMS — making rigorous, hands-on learning accessible, measurable, and scalable for institutions of any size.

Key differentiators:

- Seamless simulation integration within standard course modules — no separate portals or logins.
- Real-time simulation scoring automatically feeding the gradebook.
- Competency and learning-outcome mapping at the simulation step level.
- Multi-tenant architecture supporting multiple institutions from a single deployment.
- Full LTI 1.3 and SCORM 2004 compliance for external simulation and content integration.

2. Overall Description

2.1 Product Perspective

SimuLearn operates as a multi-tenant SaaS platform deployed on cloud infrastructure. It integrates with:

- External simulation engines (custom WebGL/Three.js labs, Unity WebGL exports, PhET simulations) via an LTI 1.3 / REST bridge.
- Identity providers via OAuth 2.0 / SAML 2.0 SSO for institutional authentication.
- Third-party storage services (AWS S3 or compatible) for media and large content files.
- Email and push notification services for user communication.
- Analytics data warehouses via event streaming for institution-level reporting.

2.2 Product Functions — High-Level Summary

Function Area	Core Capabilities
Identity & Access	Registration, login, SSO, MFA, RBAC, role assignment, session management.
Institution Management	Multi-tenant institutions, departments, domains, academic terms.
Course Management	Course creation, modules, lessons, content upload, enrollment management, publishing.
Simulation System	Virtual lab catalog, simulation player, session tracking, step events, auto-scoring.
Assessment & Grading	Assignments, quizzes, question banks, rubrics, gradebook, grade export.
Communication	Announcements, discussion forums, direct messaging, email/push notifications.
Progress & Analytics	Student dashboards, competency tracking, instructor reports, admin analytics.
Content Management	File storage, rich-text editor, SCORM/xAPI packages, video hosting.
Administration	User management, platform settings, audit logs, subscription/billing management.

2.3 Operating Environment

Component	Specification
Frontend	React.js (TypeScript) SPA — Chrome 100+, Firefox 100+, Safari 15+, Edge 100+, iOS Safari, Android Chrome.

Backend	Node.js + Express or Python FastAPI — RESTful API with JWT authentication.
Database	PostgreSQL (primary relational store) + Redis (caching/sessions).
File Storage	AWS S3 or compatible object storage with CDN distribution.
Simulation Runtime	Browser-based (WebGL, Canvas, WASM). No client-side plugin required.
Deployment	Docker-based, orchestrated with Kubernetes on AWS/GCP/Azure.
Minimum Client	1.5 Mbps internet, 4 GB RAM, modern GPU for 3D simulations.

2.4 Design Constraints

- All simulations must run in-browser without installing plugins or native apps.
- The platform must be WCAG 2.1 AA accessible.
- Data residency requirements for EU institutions must be configurable (GDPR compliance).
- API must be versioned (v1, v2) for backward compatibility during upgrades.
- No vendor lock-in for simulation content — support SCORM 2004, xAPI, and LTI 1.3.

2.5 Assumptions and Dependencies

- Institutions provide valid domain names and SMTP credentials for email delivery.
- Simulation content is pre-built or externally hosted; the platform handles launch, tracking, and scoring.
- Third-party payment processor (e.g., Stripe) is used for subscription billing; no PCI data is stored on-platform.
- All uploaded content is assumed copyright-cleared by the uploader; the platform logs but does not verify.

3. User Roles and Responsibilities

SimuLearn employs a hierarchical Role-Based Access Control (RBAC) model. Each role is described below with its scope, key capabilities, and typical persona.

Role	Scope	Key Responsibilities
Super Admin	Platform-level	Manages all institutions, platform configuration, system health, global analytics, billing, and support. Has unrestricted access to all data and settings.
Institution Admin	Institution-level	Manages all users, courses, departments, domains, roles, and settings within their assigned institution. Cannot access other institutions' data.
Department / Domain Manager	Domain-level	Oversees courses, instructors, and students within an assigned department or domain. Approves course publishing, manages domain-level analytics.
Instructor	Course-level	Creates and manages courses, modules, lessons, simulations, assignments, and quizzes. Grades student work, communicates with students.
Teaching Assistant	Course-level (limited)	Assists instructors with grading, discussions, and announcements. Cannot modify course structure or publish/unpublish courses.
Student	Course-level	Enrolls in courses, accesses content, launches simulations, submits assignments, takes quizzes, tracks personal progress.
Content Creator	Content library	Creates and manages reusable learning objects (videos, documents, SCORM packages, interactive content) for use across courses.
Simulation Developer	Simulation catalog	Builds, imports, and maintains virtual simulations; defines metadata, learning objectives, and scoring parameters.
Guest / Demo User	Public catalog only	Accesses a limited set of demo simulations and public course previews without authentication. Cannot submit work or earn grades.

4. Functional Requirements

Requirements are classified by priority: Must (essential for MVP), Should (important but not blocking launch), Could (desirable for future sprints).

4.1 Authentication and Authorization

ID	Name	Description	User Role	Priority	Acceptance Criteria
AUTH-01	User Registration	New users can register with email and password. Email verification required before login.	Student, Instructor	Must	User receives verification email; cannot log in until verified; duplicate email rejected with error message.
AUTH-02	User Login	Authenticated users log in with email/password. JWT issued on success.	All roles	Must	Valid credentials issue JWT + refresh token; invalid credentials return 401; account locked after 5 failed attempts.
AUTH-03	OAuth SSO	Users authenticate via institution's SSO (Google Workspace, Microsoft Azure AD, SAML 2.0).	All roles	Must	SSO token validated; user created/linked on first login; session created successfully.
AUTH-04	Multi-Factor Authentication	Users can enable TOTP-based MFA. Admins can enforce MFA for specific roles.	All roles	Should	MFA setup guided by QR code; login blocked until valid OTP entered when MFA enabled.
AUTH-05	Password Reset	Users can request a password reset email with a time-limited secure link.	All roles	Must	Reset link expires in 24 hours; password updated; old sessions invalidated.
AUTH-06	Role-Based Access Control	Every API endpoint and UI route is guarded by role/permission checks.	System	Must	Unauthorized requests return 403; UI elements hidden for unauthorized roles.
AUTH-07	Session Management	JWT refresh tokens allow sessions up to 30 days. Users can view and revoke active sessions.	All roles	Should	Refresh token rotation on each use; revoked tokens rejected immediately.
AUTH-08	Audit Login Events	All login, logout, and failed attempts are logged with IP, timestamp, and user agent.	Super Admin, Institution Admin	Must	Audit log queryable; failed logins visible in security dashboard.

4.2 User Management

ID	Name	Description	User Role	Priority	Acceptance Criteria
USR-01	Create Users	Admins can create user accounts individually or via CSV bulk import.	Institution Admin,	Must	Single user form validates required fields; CSV import

			Super Admin		reports success/failure per row.
USR-02	Edit Users	Admins edit user profile fields, status (active/suspended), and role assignments.	Institution Admin, Super Admin	Must	Changes saved immediately; user notified by email.
USR-03	Deactivate/Delete Users	Admins can suspend or permanently delete users. Deletion anonymizes historical data.	Institution Admin, Super Admin	Must	Suspended users cannot log in; deletion confirmed by admin; anonymization preserves grade records.
USR-04	User Profile	Each user has a profile with name, avatar, bio, institution, department, and contact details.	All roles	Must	Profile fields update in real time; avatar upload supports JPG/PNG up to 2 MB.
USR-05	Impersonation	Super Admins can impersonate any user for support purposes; actions logged.	Super Admin	Should	Impersonation session clearly labeled; all actions appear in audit log as impersonated.
USR-06	Bulk Enrollment	Admins can enroll groups of students in courses by uploading a CSV or selecting a cohort.	Institution Admin, Instructor	Must	Enrollment transactions atomic; errors reported per student; confirmation email sent.

4.3 Role and Permission Management

ID	Name	Description	User Role	Priority	Acceptance Criteria
PERM-01	Define Permissions	Super Admins define atomic permissions (e.g., course.create, grade.edit).	Super Admin	Must	Permission list versioned; changes propagate to all role assignments.
PERM-02	Create Custom Roles	Institution Admins can define custom roles by selecting permission sets.	Institution Admin	Should	Custom role scoped to institution; cannot exceed creator's own permission level.
PERM-03	Assign Roles	Admins assign roles to users per institution or per course context.	Institution Admin, Instructor	Must	Role assignment takes effect within current session; logged in audit trail.
PERM-04	Role Inheritance	Roles inherit permissions from parent roles (e.g., Instructor inherits Student permissions).	System	Should	Effective permission set computed at login; displayed in user profile.

4.4 Institution and Department Management

ID	Name	Description	User Role	Priority	Acceptance Criteria
INST-01	Create Institution	Super Admin creates a new tenant institution with name, domain, logo, and settings.	Super Admin	Must	Institution provisioned with isolated data space; Admin user created; white-label settings applied.

INST-02	Academic Terms	Admins define academic terms/semesters with start/end dates.	Institution Admin	Must	Courses can be scoped to a term; term boundaries affect enrollment windows and grade rollups.
INST-03	Department CRUD	Admins create, edit, and deactivate departments under their institution.	Institution Admin	Must	Department linked to courses and users; deactivation hides department from new assignments.
INST-04	Domain Management	Admins define learning domains (e.g., Physics, Biology, Engineering) that group courses and simulations.	Institution Admin, Department Manager	Must	Domains used for catalog filtering and analytics grouping.
INST-05	Institution Settings	Admins configure branding, default language, timezone, enrollment policies, and notification preferences.	Institution Admin	Must	Settings applied platform-wide for the tenant; override per-course where applicable.

4.5 Course Management

ID	Name	Description	User Role	Priority	Acceptance Criteria
CRS-01	Create Course	Instructors create a course with title, description, domain, tags, thumbnail, and visibility setting.	Instructor	Must	Course created in draft state; appears in instructor's dashboard; not visible to students until published.
CRS-02	Publish/Unpublish Course	Instructors publish courses to make them visible to enrolled students; can unpublish to suspend access.	Instructor, Dept Manager	Must	Published state change triggers notification to enrolled students.
CRS-03	Clone Course	Instructors clone an existing course to reuse structure and content for a new term.	Instructor	Should	Cloned course has independent content copies; shared simulations linked by reference.
CRS-04	Enrollment Settings	Instructors configure open enrollment, approval-required enrollment, enrollment code, or admin-only enrollment.	Instructor	Must	Enrollment mode enforced at all entry points; enrollment cap configurable.
CRS-05	Course Catalog	All published, enrollable courses are displayed in a searchable, filterable catalog.	Student, Guest	Must	Filtering by domain, level, instructor, format (simulations/no simulations); search by keyword.
CRS-06	Archive Course	Instructors and admins archive completed courses; archived courses are read-only.	Instructor, Admin	Must	Archived courses excluded from active catalog; grade data preserved.
CRS-07	Import/Export Course	Courses can be exported and re-imported as	Instructor, Admin	Could	Export includes all modules, lessons, assignment specs (not

		structured packages (ZIP with metadata JSON).			student data); import creates a new draft.
--	--	---	--	--	--

4.6 Module and Lesson Management

ID	Name	Description	User Role	Priority	Acceptance Criteria
MOD-01	Create Module	Instructors add ordered modules within a course.	Instructor	Must	Modules support prerequisites (unlock on prior completion); drag-and-drop reorder.
MOD-02	Create Lesson	Instructors add lessons within a module; lesson types: rich text, video, file, external URL, embedded simulation.	Instructor	Must	Lesson content saved as draft until published; preview mode available.
MOD-03	Lesson Completion	Students mark lessons complete or system auto-marks based on time-on-page / video completion.	Student	Must	Completion recorded in progress tracker; unlocks next item if prerequisites configured.
MOD-04	Content Ordering	Instructors reorder modules and lessons via drag-and-drop.	Instructor	Must	Order change reflected immediately for enrolled students.
MOD-05	Module Locking	Modules can be locked until a release date or until prerequisite modules are completed.	Instructor	Should	Locked module shows label and unlocks automatically at configured date/time.

4.7 Virtual Simulation Catalog

ID	Name	Description	User Role	Priority	Acceptance Criteria
SIM-01	Browse Simulation Catalog	All users can browse simulations filtered by domain, level, topic, and type.	All roles	Must	Catalog shows title, description, thumbnail, domain, estimated time, and rating.
SIM-02	Simulation Metadata	Each simulation has: title, description, learning objectives, prerequisites, subject domain, difficulty level, estimated duration, supported languages.	Simulation Developer	Must	Metadata searchable; learning objectives mapped to course outcomes.
SIM-03	Add Simulation to Course	Instructors add catalog simulations to a course module as a graded or ungraded activity.	Instructor	Must	Simulation appears as a lesson item; grading weight configurable; attempt limit settable.
SIM-04	Simulation Preview	Instructors can launch a simulation in preview mode without creating a student session.	Instructor, Simulation Developer	Must	Preview mode not tracked or scored; watermarked with "Preview" indicator.

SIM-05	Custom Simulation Upload	Simulation Developers upload new simulations via SCORM 2004 package or LTI 1.3 tool registration.	Simulation Developer	Must	Validation checks SCORM manifest; LTI registration tested with handshake confirmation.
--------	--------------------------	---	----------------------	-------------	--

4.8 Simulation Player and Launcher

ID	Name	Description	User Role	Priority	Acceptance Criteria
PLR-01	Launch Simulation	Enrolled students launch a simulation from within a course module.	Student	Must	Simulation loads within 5 seconds on 5 Mbps connection; session ID created; start event logged.
PLR-02	Full-Screen Mode	Students can toggle full-screen simulation view.	Student	Must	Full-screen toggle available; works on all supported browsers.
PLR-03	Save & Resume	Simulation progress auto-saved every 30 seconds; students can resume an incomplete session.	Student	Must	Resuming loads last checkpoint; elapsed time continues from prior session.
PLR-04	Simulation Instructions Panel	An expandable side panel shows step-by-step lab instructions alongside the simulation.	Student	Must	Instructions synchronized with simulation steps; collapsible to maximize simulation canvas.
PLR-05	Hint System	Students can request hints during a simulation step; hint usage logged and may affect score.	Student	Should	Hints revealed incrementally; hint usage deducted from step score per configured penalty.
PLR-06	Simulation Completion	On completion, a summary screen shows score, competencies mastered, and time taken.	Student	Must	Completion event sent to grade book; certificate trigger evaluated; attempt count incremented.

4.9 Simulation Session Tracking

ID	Name	Description	User Role	Priority	Acceptance Criteria
SST-01	Event Logging	All student interactions (step entry, tool use, data input, hint requests, step completion) logged with timestamp.	System	Must	Events stored per session; retrievable by instructor for detailed analysis.
SST-02	Attempt History	Students and instructors can view a timeline of all simulation attempts with score, date, and duration.	Student, Instructor	Must	Attempt list paginated; instructor can see all students' attempts for a course simulation.
SST-03	Score Calculation	Final simulation score calculated from step scores, hint penalties, and attempt-count modifiers.	System	Must	Scoring formula configurable per simulation; maximum possible score displayed.
SST-04	Competency Mapping	Each completed step mapped to one or more	System	Should	Competency dashboard reflects cumulative

		learning competencies; proficiency level updated.			performance across all simulations.
SST-05	Time Tracking	Total active time per simulation session recorded, excluding idle periods (>5 min inactivity = pause).	System	Must	Time data visible to instructors; included in analytics reports.

4.10 Assignments

ID	Name	Description	User Role	Priority	Acceptance Criteria
ASGN-01	Create Assignment	Instructors create assignments with title, description, due date, point value, and submission type.	Instructor	Must	Submission types: file upload, text entry, URL, simulation-linked. Rubric attachable.
ASGN-02	Submit Assignment	Students submit assignments before the due date via the configured submission type.	Student	Must	Confirmation shown; late submission flag applied if after due date; original submission preserved on resubmit.
ASGN-03	Group Assignment	Instructors designate assignments as group work; one submission per group is graded.	Instructor	Should	Group members all receive the grade; group formation tool provided.
ASGN-04	Grade Assignment	Instructors grade submissions with a numeric score, rubric scoring, or pass/fail.	Instructor, TA	Must	Grade recorded in gradebook; student notified; instructor feedback saved with submission.
ASGN-05	Late Submission Policy	Instructors configure late submission acceptance, automatic score deduction percentage, or hard cutoff.	Instructor	Should	Late deduction applied at grading time; flagged in gradebook.
ASGN-06	Plagiarism Flag	Instructors can flag a submission for plagiarism review; flagged submissions excluded from grade roll-up until resolved.	Instructor	Could	Flag visible to admin; status tracked; student not notified until admin decision.

4.11 Quizzes and Exams

ID	Name	Description	User Role	Priority	Acceptance Criteria
QZ-01	Create Quiz	Instructors create quizzes with title, instructions, time limit, attempt limit, and due window.	Instructor	Must	Quiz can draw from question bank; question order and answer order randomizable.
QZ-02	Question Types	Supported types: Multiple Choice, Multiple Select, True/False, Short Answer,	Instructor	Must	Each type renders correctly; auto-graded types scored on submission; essay types queued for manual grading.

		Numeric, Fill-in-Blank, Matching, Essay.			
QZ-03	Question Bank	Instructors and Content Creators build a reusable question bank organized by domain, topic, and difficulty.	Instructor, Content Creator	Must	Questions tagged for search; used across multiple quizzes; versioned on edit.
QZ-04	Quiz Submission	Students submit quizzes within the time limit; timer visible; auto-submitted on expiry.	Student	Must	Timer displayed prominently; auto-save every 60 seconds; submission confirmation shown.
QZ-05	Auto-Grading	Objective questions graded instantly on submission; results visible per configuration (immediate or after due date).	System	Must	Score and correct/incorrect feedback shown; essay questions marked as pending manual grading.
QZ-06	Quiz Analytics	Instructors see per-question statistics: average score, most common wrong answer, difficulty index.	Instructor	Should	Statistics available after at least 5 submissions; exportable as CSV.
QZ-07	Accommodations	Instructors can grant individual students extended time or additional attempts.	Instructor	Should	Accommodation overrides stored per student per quiz; applied automatically at launch.

4.12 Gradebook

ID	Name	Description	User Role	Priority	Acceptance Criteria
GB-01	View Gradebook	Instructors view a grid of all students vs. all graded activities with scores and totals.	Instructor, TA	Must	Columns sortable and filterable; missing grades flagged; late submissions indicated.
GB-02	Grade Calculation	Course total grade calculated from configurable category weights (Assignments 40%, Quizzes 30%, Simulations 30%).	Instructor	Must	Weights sum to 100%; grade automatically recalculated on each graded submission.
GB-03	Grade Schemes	Support letter grades, percentage, pass/fail, and custom scale grade schemes.	Instructor	Must	Grade scheme applied to final grade display; configurable per course.
GB-04	Export Grades	Instructors export full gradebook as CSV or Excel.	Instructor, Admin	Must	Export includes student ID, name, all activity scores, and course total.
GB-05	Student Grade View	Students see their own grades, feedback, and course standing.	Student	Must	Student view shows assignment scores, quiz scores, simulation scores, and cumulative grade.

GB-06	Grade Override	Instructors can manually override a system-calculated grade with justification note.	Instructor	Must	Override logged; original score preserved; student notified.
-------	----------------	--	------------	-------------	--

~~4.13~~ Rubrics

ID	Name	Description	User Role	Priority	Acceptance Criteria
RUB-01	Create Rubric	Instructors create rubrics with criteria rows and performance level columns.	Instructor	Must	Rubric builder with point values per cell; total points calculated automatically.
RUB-02	Attach Rubric	Rubric attached to an assignment; graders select cells to generate score.	Instructor	Must	Rubric score maps to grade; criterion-level feedback saveable per cell.
RUB-03	Reuse Rubric	Institution rubric library allows sharing rubrics across courses and instructors.	Instructor, Content Creator	Should	Rubric cloned for course-level customization; original unchanged.

~~4.14~~ Learning Outcomes and Competencies

ID	Name	Description	User Role	Priority	Acceptance Criteria
OUT-01	Define Outcomes	Institution Admins and Instructors define learning outcomes/competencies at institution or course level.	Institution Admin, Instructor	Must	Outcomes organized in a hierarchy; tagged with Bloom's taxonomy level.
OUT-02	Map to Activities	Outcomes mapped to simulation steps, quiz questions, and assignment criteria.	Instructor, Simulation Developer	Must	Many-to-many mapping; evidence collected automatically from graded activities.
OUT-03	Mastery Tracking	System tracks each student's proficiency level per competency based on weighted evidence.	System	Should	Proficiency levels: Not Yet, Developing, Proficient, Mastered; visualized in student dashboard.
OUT-04	Institution Outcomes Report	Aggregate outcome achievement reported at domain, department, and institution level.	Institution Admin, Dept Manager	Should	Report shows percentage of students at each proficiency level per outcome.

4.15 Progress Tracking

ID	Name	Description	User Role	Priority	Acceptance Criteria
PROG-01	Course Progress	Student's completion percentage per course calculated from completed	Student, Instructor	Must	Progress bar visible on course card and course header; real-time update on activity completion.

		lessons, simulations, and graded items.			
PROG-02	Activity Completion	Each lesson, simulation, quiz, and assignment has a completion status (Not Started, In Progress, Completed, Passed, Failed).	Student	Must	Completion status rules configurable per item type (e.g., score >= 70% = Passed).
PROG-03	Learning Streak	Students earn a learning streak badge for daily engagement; instructors can view streaks.	Student	Could	Streak resets after 24-hour gap; longest streak displayed on profile.
PROG-04	Pace Alerts	System alerts instructors when a student falls significantly behind course pace.	Instructor	Should	Configurable threshold (e.g., 2+ weeks behind); email and dashboard alert generated.

~~4.16~~ Calendar

ID	Name	Description	User Role	Priority	Acceptance Criteria
CAL-01	Course Calendar	Assignments, quizzes, simulation deadlines, and announcements appear on a course and personal calendar.	All roles	Must	Calendar views: month, week, day, agenda; iCal export supported.
CAL-02	Event Creation	Instructors add events and office hours to the course calendar.	Instructor	Must	Events support title, description, location (URL or physical), start/end time, and recurrence.

~~4.17~~ Announcements

ID	Name	Description	User Role	Priority	Acceptance Criteria
ANN-01	Create Announcement	Instructors post announcements to all course participants with optional immediate notification.	Instructor	Must	Announcement appears in course feed and student dashboard; email notification configurable.
ANN-02	Pin Announcement	Instructors pin important announcements to the top of the course feed.	Instructor	Must	Pinned announcements displayed first; maximum 3 pinned per course.

~~4.18~~ Discussions

ID	Name	Description	User Role	Priority	Acceptance Criteria
----	------	-------------	-----------	----------	---------------------

DISC-01	Create Discussion	Instructors create graded or ungraded discussion topics within course modules.	Instructor	Must	Topic has prompt, participation requirements, and optional due date.
DISC-02	Student Post/Reply	Students post initial responses and reply to peers; rich-text with file attachments supported.	Student	Must	Post character minimum configurable; notification to instructor on first post.
DISC-03	Moderate Discussion	Instructors and TAs can edit, delete, or lock individual posts.	Instructor, TA	Must	Moderated posts flagged; original content soft-deleted (recoverable by admin).

4.19 Messaging and Notifications

ID	Name	Description	User Role	Priority	Acceptance Criteria
MSG-01	Direct Messaging	Users send direct messages to other users within the platform.	All roles	Must	Inbox with conversation threads; file attachments up to 10 MB; message search.
MSG-02	Notification Center	In-platform notification bell aggregates all alerts (grades, announcements, messages, due dates).	All roles	Must	Notification count badge; mark all read; notification preferences per type.
MSG-03	Email Notifications	Configurable email notifications for key events (assignment due, grade posted, new announcement).	All roles	Must	User sets notification frequency (instant, daily digest, weekly digest, off).
MSG-04	Push Notifications	Browser push notifications for time-sensitive events (exam starting, assignment overdue).	Student, Instructor	Should	Permission requested on first login; requires HTTPS; user can revoke.

4.20 Certificates

ID	Name	Description	User Role	Priority	Acceptance Criteria
CERT-01	Certificate Template	Institution Admins create certificate templates with institution branding, fields, and signature images.	Institution Admin	Should	Template editor supports logo, text fields, signatures; preview before saving.
CERT-02	Auto-Issue Certificate	Certificates issued automatically when a student meets course completion criteria.	System	Should	Criteria: course grade $\geq$ threshold AND all required items completed.
CERT-03	Verify Certificate	Each certificate has a unique, publicly verifiable URL showing authenticity.	Guest	Should	Verification page shows recipient, course, institution, date, and validity status.

4.21 Admin Dashboard

ID	Name	Description	User Role	Priority	Acceptance Criteria
ADSH-01	Platform Overview	Super Admin sees real-time metrics: total institutions, users, active sessions, storage used, API health.	Super Admin	Must	Dashboard refreshes every 60 seconds; clickable tiles drill down to detail views.
ADSH-02	Institution Dashboard	Institution Admin sees enrollment counts, active courses, recent activity, and pending approvals.	Institution Admin	Must	Key metrics summarized; quick links to user management and course management.
ADSH-03	System Health Monitor	Super Admin views server uptime, API response times, error rate, and queue depth.	Super Admin	Should	Alerts triggered if error rate > 1% or response time > 2 s; page includes historical graphs.

4.22 Reports and Analytics

ID	Name	Description	User Role	Priority	Acceptance Criteria
RPT-01	Student Progress Report	Per-student breakdown of course completion, grades, simulation performance, and time-on-platform.	Instructor, Admin	Must	Filterable by course, date range, and student cohort; exportable as PDF or CSV.
RPT-02	Course Completion Report	Aggregate completion rates, average grades, and dropout points for each course.	Instructor, Dept Manager, Admin	Must	Funnel visualization from enrollment to completion.
RPT-03	Simulation Performance Report	Per-simulation analytics: average score, average time, most failed steps, hint usage rates.	Instructor, Simulation Developer	Must	Step-level breakdown; comparison across student cohorts.
RPT-04	Institution Analytics Dashboard	Top-level KPIs for institution: active users, course count, simulation hours, outcome achievement.	Institution Admin, Super Admin	Must	Trend charts (30/60/90 days); domain-level drilldown.
RPT-05	Engagement Report	Daily/weekly active users, session length distribution, feature usage heatmap.	Institution Admin, Super Admin	Should	Filterable by role, department, and date range.
RPT-06	Export All Reports	All reports exportable to CSV, Excel, or PDF.	Instructor, Admin	Must	Export captures current filter state; large exports queued and emailed as download link.

4.23 Content Management

ID	Name	Description	User Role	Priority	Acceptance Criteria
----	------	-------------	-----------	----------	---------------------

CMS-01	File Upload	Users upload files (PDF, DOCX, PPTX, MP4, MP3, JPG, PNG, ZIP) up to 500 MB per file.	Instructor, Content Creator	Must	Virus scan on upload; MIME type validation; progress indicator; CDN delivery after upload.
CMS-02	Rich Text Editor	Lesson content editable with a rich-text editor supporting formatting, embedded media, equations (LaTeX), and code blocks.	Instructor, Content Creator	Must	Editor saves auto-drafts every 30 seconds; version history stored.
CMS-03	SCORM Package Import	Import SCORM 2004 / SCORM 1.2 packages as lesson items with tracking.	Instructor, Content Creator	Must	SCORM manifest parsed; completion and score events tracked via SCORM API.
CMS-04	Institution Media Library	Shared media library per institution for reusable assets across courses.	Institution Admin, Content Creator	Should	Assets tagged, searched, and organized in folders; usage count tracked.

4.24 API and Integrations

ID	Name	Description	User Role	Priority	Acceptance Criteria
API-01	REST API	Platform exposes a versioned REST API for all major resources.	Developers	Must	API follows OpenAPI 3.0 specification; Swagger UI available at /api/docs.
API-02	Webhook Support	Platform emits webhooks for key events (enrollment, grade posted, simulation completed).	Developers	Should	Webhooks configurable per institution; HMAC-signed payloads; retry on failure.
API-03	LTI 1.3 Provider	Platform acts as an LTI 1.3 Tool Consumer and Tool Provider for external tools.	Simulation Developer, Admin	Must	LTI launch handshake validated; grade passback to LTI platform supported.
API-04	Calendar Sync	Students and instructors sync course calendar to Google Calendar or Outlook via iCal feed.	Student, Instructor	Should	iCal URL per user per course; auto-updates on event changes.
API-05	Payment API Integration	Stripe (or equivalent) integration for subscription billing and per-course payment.	Super Admin	Could	Payment intents processed server-side; no card data touches platform servers.

4.25 Audit Logs and System Settings

ID	Name	Description	User Role	Priority	Acceptance Criteria
AUDIT-01	Audit Log	All create/update/delete operations on sensitive entities logged with actor, action, entity, timestamp, and delta.	Super Admin, Institution Admin	Must	Logs immutable; searchable by date, user, entity type; exportable.

AUDIT-02	System Settings	Super Admin configures global settings: maintenance mode, feature flags, storage limits, email provider, rate limits.	Super Admin	Must	Settings UI with validation; changes logged; rollback available for most settings.
AUDIT-03	Maintenance Mode	Super Admin enables maintenance mode to block user login during updates.	Super Admin	Must	Maintenance page shown to all non-admin users; in-flight simulation sessions warned 10 min before activation.

5. Non-Functional Requirements

ID	Requirement	Description	Priority
NFR-01	Performance	API p95 response time < 300 ms under 1,000 concurrent users. Simulation initial load time < 5 seconds on a 5 Mbps connection. Page renders (LCP) < 2.5 seconds.	Must
NFR-02	Scalability	System horizontally scalable to support 100,000 concurrent users. Kubernetes HPA auto-scales pods based on CPU > 70%.	Must
NFR-03	Availability	99.9% monthly uptime SLA for production. Planned maintenance windows < 4 hours/month, scheduled off-peak. Zero-downtime deployments via rolling updates.	Must
NFR-04	Security	All data encrypted in transit (TLS 1.3+). Sensitive data encrypted at rest (AES-256). Quarterly penetration testing. OWASP Top 10 addressed in CI/CD pipeline. Rate limiting on all public API endpoints.	Must
NFR-05	Privacy	GDPR compliant for EU users: right to access, portability, and erasure. FERPA compliant for US educational records. Configurable data retention periods. Consent management for analytics cookies.	Must
NFR-06	Accessibility	WCAG 2.1 Level AA compliance for all platform UI. Simulations should meet WCAG 2.1 AA where technically feasible; documented exceptions for 3D interactive components.	Must
NFR-07	Compatibility	Supports Chrome 100+, Firefox 100+, Safari 15+, Edge 100+. Mobile-responsive (320px to 2560px). iOS Safari 15+ and Chrome Android 100+ for mobile learning.	Must
NFR-08	Maintainability	Backend code coverage >= 80%. All API endpoints documented in OpenAPI 3.0. Comprehensive README and developer setup guide. Changelog maintained per release.	Should
NFR-09	Usability	New student onboarding completable in < 5 minutes. First simulation launch <= 3 clicks from course page. SUS (System Usability Scale) score target >= 75.	Must
NFR-10	Reliability	RTO (Recovery Time Objective) < 4 hours. RPO (Recovery Point Objective) < 1 hour. Database replication with automatic failover in < 30 seconds.	Must
NFR-11	Localization	Platform supports UTF-8 throughout. UI available in English (default), Arabic (RTL), French, Spanish, and German at launch. Additional languages addable via i18n resource files.	Should
NFR-12	Backup & Recovery	Automated daily full backups and hourly incremental backups of all databases. Backups retained 30 days. Backup restoration tested monthly.	Must
NFR-13	Logging & Monitoring	Centralized structured logging (ELK or equivalent). Application performance monitoring (Datadog or New Relic). Alerting for error rate > 1% or p99 latency > 1 s.	Must

6. Use Cases

UC-01: Student Registration and Login

Actor	Student / New User
Goal	Create an account and gain access to the platform.
Preconditions	Public registration is enabled for the institution. User has a valid email.

Main Flow:

1. User navigates to the registration page.
2. User enters first name, last name, email, and password.
3. System sends a verification email.
4. User clicks the verification link.
5. Account is activated; user is redirected to the student dashboard.

Alternative Flow:

- SSO Login: User clicks "Sign in with Google/Microsoft" — OAuth flow redirects and creates/links account automatically.
- Enrollment Code: User enters a course enrollment code during registration and is auto-enrolled.

Postconditions:

User account created and verified. User is logged in with a valid JWT session.

Exceptions:

Invalid email format returns form validation error. Duplicate email shows "Email already registered" message.

UC-02: Student Enrolling in a Course

Actor	Student
Goal	Enroll in an available course to access its content.
Preconditions	Student is authenticated. Course is published and accepting enrollments.

Main Flow:

1. Student browses the course catalog.
2. Student clicks on a course to view the course details page.
3. Student clicks "Enroll Now".

9. 4. If open enrollment: enrollment confirmed immediately.
10. 5. Student is redirected to the course homepage.
11. 6. Course appears in student's "My Courses" list.

Alternative Flow:

- Approval-required enrollment: student submits enrollment request; instructor receives notification; student enrolled only after approval.
- Paid course: student directed to payment flow before enrollment confirmed.

Postconditions:

Student enrolled in course. Progress tracking initialized at 0%. Course appears in dashboard.

Exceptions:

Course is full: "Enrollment full" message shown. Instructor is not accepting new enrollments: explanation shown.

UC-03: Student Starting a Virtual Simulation

Actor	Student
Goal	Launch and begin an assigned virtual simulation.
Preconditions	Student is enrolled in the course. Simulation is assigned in an accessible module.

Main Flow:

12. 1. Student navigates to the course module containing the simulation.
13. 2. Student clicks on the simulation activity.
14. 3. Platform shows simulation description, learning objectives, and attempt history.
15. 4. Student clicks "Start Simulation".
16. 5. Platform creates a new simulation session record.
17. 6. Simulation iframe/engine loads within the player page.
18. 7. Instructions panel opens alongside the simulation.

Alternative Flow:

- Resume previous attempt: if an incomplete session exists, student is prompted to resume or start fresh.
- Attempt limit reached: "Start" button replaced with "View Last Attempt"; student cannot start new session.

Postconditions:

Simulation session created. Start event logged. Timer begins.

Exceptions:

Simulation engine fails to load: error page with retry option shown; session not created.

UC-04: Student Completing a Simulation-Based Lab

Actor	Student
Goal	Complete all steps of a simulation and receive a grade.
Preconditions	Active simulation session exists. Student is within an incomplete simulation.

Main Flow:

19. 1. Student progresses through experiment steps, interacting with simulation tools.
20. 2. System logs each step interaction event.
21. 3. Student completes all required steps.
22. 4. Simulation sends completion event to the platform.
23. 5. Platform calculates final score from step scores and hint penalties.
24. 6. Score is written to the gradebook.
25. 7. Completion summary screen shows score, time taken, and competencies earned.

Alternative Flow:

- Partial completion: student exits mid-simulation; session saved; progress preserved for resume.
- Retake: student submits and immediately starts a new attempt (if attempts remaining).

Postconditions:

Simulation session marked Complete. Grade recorded. Competency levels updated. Certificate trigger evaluated.

Exceptions:

Score calculation fails: grade marked "Pending Review"; instructor notified for manual grading.

UC-05: Student Submitting an Assignment

Actor	Student
Goal	Upload or enter an assignment submission before the due date.
Preconditions	Student is enrolled. Assignment is published and within the submission window.

Main Flow:

26. 1. Student navigates to the assignment page.
27. 2. Student reads the instructions and downloads any provided materials.

28. 3. Student prepares their response (file, text, or URL).
29. 4. Student clicks "Submit Assignment".
30. 5. Student attaches file or enters text in the submission field.
31. 6. Student clicks "Submit"; confirmation dialog shown.
32. 7. Submission saved with timestamp; confirmation email sent.

Alternative Flow:

- Late submission: after due date, if late submissions allowed, a late flag is applied and score deduction noted.
- Resubmission: if allowed by instructor, student replaces prior submission; original preserved in history.

Postconditions:

Submission recorded. Instructor notified. Submission visible in gradebook as "Submitted".

Exceptions:

File exceeds 500 MB limit: upload rejected with size limit message. File type not allowed: validation error shown.

UC-06: Student Taking a Quiz

Actor	Student
Goal	Complete a timed quiz and receive an auto-graded score.
Preconditions	Student is enrolled. Quiz is available within its window. Attempts remaining.

Main Flow:

33. 1. Student navigates to the quiz in the course module.
34. 2. Student reads instructions including time limit and attempt count.
35. 3. Student clicks "Begin Quiz".
36. 4. Attempt record created; timer starts.
37. 5. Questions displayed one-at-a-time or all-at-once per instructor setting.
38. 6. Student answers questions; auto-save every 60 seconds.
39. 7. Student submits quiz before timer expires.
40. 8. System auto-grades objective questions; essay questions queued.
41. 9. Score (partial if essays pending) shown immediately.

Alternative Flow:

- Timer expiry: quiz auto-submitted at time limit; all unanswered questions scored as 0.
- Navigation away warning: browser displays unload warning; session preserved if student returns within 5 minutes.

Postconditions:

Quiz attempt recorded. Auto-graded score added to gradebook. Student notified.

Exceptions:

Network disconnection during quiz: responses cached locally; reconnection resumes from last saved state.

UC-07: Instructor Creating a Course

Actor	Instructor
Goal	Set up a new course with structure and content.
Preconditions	Instructor authenticated. Has course.create permission.

Main Flow:

42. 1. Instructor navigates to "My Courses" and clicks "New Course".
43. 2. Instructor enters course title, description, subject domain, and level.
44. 3. Instructor uploads a course thumbnail.
45. 4. Instructor clicks "Create Course" — course created in Draft state.
46. 5. Instructor adds modules using "Add Module" button.
47. 6. Within each module, instructor adds lessons, simulations, assignments, and quizzes.
48. 7. Instructor configures enrollment settings.
49. 8. Instructor clicks "Publish" — course visible to eligible students.

Alternative Flow:

- Template-based creation: instructor selects a course template; pre-populated structure editable.
- Clone existing course: instructor copies a past course; all content duplicated.

Postconditions:

Course published. Enrolled students notified. Course appears in catalog per visibility settings.

Exceptions:

Publishing blocked if required fields missing (title, at least one module); validation errors shown.

UC-08: Instructor Grading Student Work

Actor	Instructor
Goal	Review and grade a student assignment submission.
Preconditions	Assignment has at least one submission. Instructor has grade.edit permission.

Main Flow:

50. 1. Instructor navigates to the assignment in the course.
51. 2. Instructor opens the "Submissions" tab.
52. 3. Instructor selects an ungraded submission.
53. 4. Submission viewer displays student work (rendered file or text).
54. 5. Instructor scores using inline rubric or direct numeric entry.
55. 6. Instructor adds written feedback.
56. 7. Instructor clicks "Save Grade".
57. 8. Grade stored; student notified.

Alternative Flow:

- Bulk speed grading: instructor moves through submissions sequentially without returning to list.
- TA grading: TA completes grades; instructor reviews and finalizes with one-click approval.

Postconditions:

Grade saved. Gradebook updated. Student notification sent.

Exceptions:

Rubric not fully completed: warning shown; instructor must confirm intent to save partial rubric.

UC-09: Admin Creating Users and Roles

Actor	Institution Admin
Goal	Create new user accounts and assign appropriate roles.
Preconditions	Admin authenticated. Has user.create and role.assign permissions.

Main Flow:

58. 1. Admin navigates to User Management.
59. 2. Admin clicks "Add User" or "Import Users (CSV)".
60. 3. For single user: Admin fills in name, email, role, and department.
61. 4. Admin clicks "Create" — account created; welcome email sent.
62. 5. For bulk import: Admin uploads CSV; system validates and shows preview.
63. 6. Admin confirms import; accounts created; import report downloaded.

Alternative Flow:

- CSV error: rows with invalid data highlighted; admin can correct and re-import.
- Role creation: admin navigates to Role Management, clicks "New Role", selects permissions, saves.

Postconditions:

User accounts created. Welcome emails sent. Users can log in.

Exceptions:

Duplicate email in CSV: row skipped with error; other rows processed successfully.

UC-10: Simulation Developer Integrating a New Simulation

Actor	Simulation Developer
Goal	Register and publish a new simulation in the platform catalog.
Preconditions	Developer has simulation.create permission. Simulation package or LTI endpoint is ready.

Main Flow:

64. 1. Developer navigates to Simulation Catalog > "Add New Simulation".
65. 2. Developer selects integration type: SCORM Package or LTI 1.3 Tool.
66. 3. For SCORM: Developer uploads ZIP package; system validates manifest.
67. 4. For LTI: Developer enters Tool URL, Client ID, and public key.
68. 5. Developer fills in simulation metadata: title, description, domain, objectives, estimated time.
69. 6. Developer maps simulation competencies to platform outcome taxonomy.
70. 7. Developer configures scoring parameters and step definitions.
71. 8. Developer clicks "Save & Test" — launches simulation in preview mode.
72. 9. Developer confirms simulation works correctly and clicks "Publish to Catalog".

Alternative Flow:

- Validation failure: SCORM manifest errors listed; developer must fix before continuing.
- LTI handshake failure: error details shown; developer checks endpoint configuration.

Postconditions:

Simulation published in catalog. Instructors can add it to courses.

Exceptions:

Package > 500 MB: upload rejected; developer must optimize or use streaming URL instead.

7. User Sequence Flows

Each sequence flow below describes the interaction chain between the actor, frontend, backend API, database, and relevant services.

7.1 Login Flow

Step	Actor / Layer	Action / Event
1	Student (Browser)	Navigates to /login. Enters email and password. Clicks "Sign In".
2	Frontend	Validates form client-side. POST /api/v1/auth/login with { email, password }.
3	Backend (Auth Service)	Validates credentials against hashed password in DB. Checks account status (active/suspended).
4	Database	Returns user record with role list and institution context.
5	Backend	Issues signed JWT (15 min TTL) + HttpOnly refresh token cookie (7–30 day TTL). Logs login event in audit_logs.
6	Frontend	Stores JWT in memory. Redirects to role-appropriate dashboard. Loads user context.
7	Notification Service	(If first login of day) triggers welcome-back notification push.

7.2 Course Enrollment Flow

Step	Actor / Layer	Action / Event
1	Student	Browses course catalog. Clicks on course. Clicks "Enroll Now".
2	Frontend	POST /api/v1/enrollments with { course_id, user_id }.
3	Backend	Checks enrollment mode (open / approval / code). Verifies course is not full. Checks no existing enrollment.
4	Database	Inserts enrollment record with status "active" (open) or "pending" (approval).
5	Backend	Returns enrollment confirmation. Initializes progress record.
6	Notification Service	Sends enrollment confirmation email to student. If approval mode: sends notification to instructor.
7	Frontend	Redirects student to course homepage. Course appears in "My Courses".

7.3 Simulation Launch Flow

Step	Actor / Layer	Action / Event
1	Student	Opens course module. Clicks simulation activity. Clicks "Start Simulation".
2	Frontend	POST /api/v1/simulation-sessions with { simulation_id, course_id, user_id }.

3	Backend	Validates attempt limit. Creates simulation_session record (status: in_progress). Generates launch token.
4	Database	Inserts session record. Returns session_id and launch_token.
5	Backend (LTI/SCORM Bridge)	If LTI: constructs LTI launch request signed with platform key. If SCORM: serves SCORM manifest URL.
6	Frontend	Renders simulation player iframe with launch URL. Opens instructions panel.
7	Simulation Engine	Loads simulation assets. Sends "simulation_start" event to backend via postMessage / API.
8	Database	Logs start_event in simulation_events table.

7.4 Simulation Completion and Score Saving Flow

Step	Actor / Layer	Action / Event
1	Simulation Engine	Student completes all steps. Engine sends completion payload: { session_id, score, step_events[] }.
2	Backend (Simulation API)	Validates session ownership. Receives completion payload.
3	Backend (Scoring Engine)	Calculates final score from step_scores, hint_penalties, and attempt_modifier. Clamps to [0, max_score].
4	Database	Updates simulation_session: status="completed", score, completed_at. Inserts step_event records.
5	Backend (Grade Sync)	Writes grade to gradebook: upserts grade record for (student, simulation_activity).
6	Backend (Competency Service)	Updates student competency proficiency levels based on step-to-competency mappings.
7	Notification Service	Sends "Simulation graded" notification to student. Triggers certificate evaluation.
8	Frontend	Receives completion response. Displays summary screen with score, time, and competencies earned.

7.5 Assignment Submission Flow

Step	Actor / Layer	Action / Event
1	Student	Opens assignment page. Attaches file or enters text. Clicks "Submit".
2	Frontend	Uploads file to presigned S3 URL (if file). Then POST /api/v1/submissions with { assignment_id, content_url text }.
3	Backend	Checks submission window (due date). Applies late flag if past due. Saves submission record.
4	Database	Inserts submission record with timestamp, content reference, and late_flag. Increments submission_count.
5	Notification Service	Sends confirmation email to student. Notifies instructor of new submission.
6	Frontend	Shows submission confirmation. "Resubmit" button appears if resubmissions allowed.

7.6 Quiz Attempt Flow

Step	Actor / Layer	Action / Event
1	Student	Opens quiz. Reads instructions. Clicks "Begin Quiz".
2	Frontend	POST /api/v1/quiz-attempts with { quiz_id, user_id }. Receives attempt_id and question set.
3	Backend	Validates attempt limit. Creates quiz_attempt record (status: in_progress). Randomizes questions/answers if configured.
4	Database	Inserts attempt record. Returns question list.
5	Frontend	Renders question UI. Starts countdown timer. Auto-saves responses every 60 s.
6	Student	Answers all questions. Clicks "Submit Quiz".
7	Frontend	POST /api/v1/quiz-attempts/{id}/submit with { responses[] }.
8	Backend (Grading Engine)	Auto-grades objective questions. Marks essay questions pending. Calculates partial score.
9	Database	Updates attempt: status="submitted", auto_score, graded_at. Inserts response records.
10	Frontend	Displays score and per-question feedback (per instructor release settings).

7.7 Instructor Course Creation Flow

Step	Actor / Layer	Action / Event
1	Instructor	Navigates to My Courses. Clicks "New Course". Fills in details. Clicks "Create".
2	Frontend	POST /api/v1/courses with course metadata. Receives course_id.
3	Backend	Creates course record (status: draft). Assigns instructor as course owner.
4	Database	Inserts course record.
5	Instructor	Adds modules and lessons. Configures enrollment settings.
6	Frontend	PATCH /api/v1/courses/{id} for each edit. POST /api/v1/modules, /api/v1/lessons.
7	Instructor	Clicks "Publish".
8	Backend	Validates required fields. Changes status to "published". Indexes course in catalog search.
9	Notification Service	Notifies enrolled students (if any) that course is now available.

7.8 Admin User Management Flow

Step	Actor / Layer	Action / Event
1	Admin	Navigates to User Management. Clicks "Import Users (CSV)". Uploads file.
2	Frontend	Sends CSV to POST /api/v1/users/bulk-import. Receives validation report.

3	Backend	Parses CSV. Validates each row (email format, role exists, department exists). Returns preview with errors.
4	Admin	Reviews preview. Confirms import.
5	Frontend	POST /api/v1/users/bulk-import/confirm with import_job_id.
6	Backend	Creates user accounts in batch. Hashes passwords. Assigns roles. Triggers welcome emails.
7	Notification Service	Sends welcome emails to all new users with login link and temporary password.
8	Frontend	Shows import completion report: created, skipped (duplicate), failed rows.

8. System Architecture

8.1 Architecture Overview

SimuLearn follows a cloud-native, microservices-oriented architecture deployed on Kubernetes. The platform is logically divided into the following layers:

Layer	Component	Technology	Responsibility
Presentation	Web SPA	React.js (TypeScript) + Vite	Student, instructor, and admin UI; responsive, WCAG 2.1 AA.
Presentation	Mobile-Responsive	TailwindCSS + PWA manifest	Progressive Web App for mobile devices.
API Gateway	API Gateway	Kong / AWS API Gateway	Rate limiting, authentication pre-check, routing, logging.
Application	Auth Service	Node.js / FastAPI	JWT issuance, OAuth, SAML, MFA, session management.
Application	Core LMS API	Node.js / FastAPI (PostgreSQL)	Courses, modules, lessons, enrollments, assignments, quizzes, grades.
Application	Simulation Service	Node.js / FastAPI	Session management, LTI/SCORM bridge, scoring engine, event ingestion.
Application	Notification Service	Node.js + Bull Queue	Email (SendGrid), push (Web Push), in-app notification delivery.
Application	Analytics Service	Python (FastAPI) + ClickHouse	Event processing, report generation, dashboard data aggregation.
Application	File Service	Node.js + AWS S3	Presigned upload URLs, CDN asset management, virus scanning.
Data	Primary Database	PostgreSQL 15 (primary + read replica)	Relational data: users, courses, grades, enrollments.
Data	Cache / Sessions	Redis 7	JWT refresh tokens, session state, API response caching.
Data	Analytics Store	ClickHouse	Time-series event data for analytics queries.
Data	Search Index	Elasticsearch / OpenSearch	Full-text search for courses, simulations, users.
Data	Object Storage	AWS S3 + CloudFront CDN	Files, media, SCORM packages, simulation assets.
Simulation	Simulation Runtime	Browser (WebGL/WASM/Canvas)	Runs simulation entirely client-side; no server compute required per student.
Simulation	LTI Bridge	LTI 1.3 Library (Node.js)	Handshake, launch token signing, grade passback.
Ops	Container Orchestration	Kubernetes (EKS/GKE)	Auto-scaling, rolling deployments, health checks.
Ops	CI/CD	GitHub Actions + ArgoCD	Test, build, deploy pipeline; staged rollouts.

Ops	Observability	Datadog / Prometheus + Grafana	Metrics, logs (ELK), traces (Jaeger), alerts.
-----	---------------	--------------------------------	---

9. Database Design

The primary database is PostgreSQL. Tables use UUID primary keys. Soft-delete (`deleted_at`) is applied to sensitive entities. The schema below covers all major entities.

users

Purpose: Stores all user accounts regardless of role.

Field	Type	Notes
id	UUID PK	Auto-generated.
email	VARCHAR(255) UNIQUE	Indexed, lowercase normalized.
password_hash	VARCHAR(255)	Bcrypt; NULL for SSO-only users.
first_name	VARCHAR(100)	
last_name	VARCHAR(100)	
avatar_url	TEXT	CDN URL.
status	ENUM(active,suspended,pending)	Default: pending until email verified.
last_login_at	TIMESTAMPTZ	
created_at / updated_at	TIMESTAMPTZ	Managed by ORM.
deleted_at	TIMESTAMPTZ	Soft-delete; NULL = active.

Relationships: Has many: `user_roles`, `enrollments`, `submissions`, `simulation_sessions`, `messages`.

institutions

Purpose: Multi-tenant institution records.

Field	Type	Notes
id	UUID PK	
name	VARCHAR(255)	
slug	VARCHAR(100) UNIQUE	URL-safe identifier.
domain	VARCHAR(255)	Email domain for SSO matching.
logo_url	TEXT	
primary_color	VARCHAR(7)	Hex color for white-labeling.
status	ENUM(active,suspended)	
settings	JSONB	Notification prefs, enrollment policies, etc.
created_at	TIMESTAMPTZ	

Relationships: Has many: `departments`, `users` (via `user_institutions`), `courses`.

roles & permissions

Purpose: RBAC role and permission definitions.

Field	Type	Notes
roles.id	UUID PK	
roles.name	VARCHAR(100)	e.g., "Student", "Instructor".
roles.institution_id	UUID FK -> institutions	NULL = system-wide role.
permissions.id	UUID PK	
permissions.code	VARCHAR(100) UNIQUE	e.g., "course.create".
role_permissions	Junction: role_id + permission_id	Many-to-many.
user_roles	Junction: user_id + role_id + context_type + context_id	Role scoped to institution or course.

Relationships: user_roles links users to roles within a specific context (institution or course).

courses

Purpose: Course definitions.

Field	Type	Notes
id	UUID PK	
title	VARCHAR(255)	
description	TEXT	
instructor_id	UUID FK -> users	Primary instructor.
institution_id	UUID FK -> institutions	
domain_id	UUID FK -> domains	
status	ENUM(draft,published,archived)	
enrollment_type	ENUM(open,approval,code,admin)	
enrollment_cap	INTEGER	NULL = unlimited.
thumbnail_url	TEXT	
settings	JSONB	Grade weights, late policy, etc.
published_at	TIMESTAMPTZ	
created_at / updated_at	TIMESTAMPTZ	

Relationships: Has many: enrollments, modules, grade_items. Belongs to: institution, domain.

modules & lessons

Purpose: Course content structure.

Field	Type	Notes
-------	------	-------

modules.id	UUID PK	
modules.course_id	UUID FK -> courses	
modules.title	VARCHAR(255)	
modules.position	INTEGER	Sort order.
modules.unlock_at	TIMESTAMPTZ	Scheduled release.
modules.prerequisite_module_id	UUID FK -> modules	Self-referential.
lessons.id	UUID PK	
lessons.module_id	UUID FK -> modules	
lessons.title	VARCHAR(255)	
lessons.type	ENUM(text,video,file,url,scorm,simulation)	
lessons.content	JSONB	Type-specific payload (URL, rich text, SCORM ref, simulation_id).
lessons.position	INTEGER	
lessons.is_required	BOOLEAN	Affects course completion.

Relationships: Lessons belong to modules; modules belong to courses.

simulations

Purpose: Virtual simulation catalog entries.

Field	Type	Notes
id	UUID PK	
title	VARCHAR(255)	
description	TEXT	
domain_id	UUID FK -> domains	
type	ENUM(scorm,lti,internal)	
launch_url	TEXT	Entry point.
thumbnail_url	TEXT	
estimated_minutes	INTEGER	
max_score	NUMERIC(5,2)	
scoring_config	JSONB	Step weights, hint penalties.
learning_objectives	TEXT[]	Array of objective strings.
competency_ids	UUID[]	Linked competencies.
status	ENUM(draft,active,deprecated)	
created_by	UUID FK -> users	Simulation Developer.

Relationships: Referenced by lessons (type=simulation). Has many simulation_sessions.

simulation_sessions

Purpose: Per-attempt simulation tracking.

Field	Type	Notes
id	UUID PK	
simulation_id	UUID FK -> simulations	
user_id	UUID FK -> users	
course_id	UUID FK -> courses	
lesson_id	UUID FK -> lessons	
status	ENUM(in_progress,completed,abandoned)	
score	NUMERIC(5,2)	Final score after completion.
max_score	NUMERIC(5,2)	Snapshot of simulation max score.
attempt_number	INTEGER	Incremented per new session.
started_at	TIMESTAMPTZ	
completed_at	TIMESTAMPTZ	
active_seconds	INTEGER	Excluding idle time.

Relationships: Has many simulation_events. Score written to grades table on completion.

assignments & submissions

Purpose: Assignment definitions and student work.

Field	Type	Notes
assignments.id	UUID PK	
assignments.course_id	UUID FK -> courses	
assignments.title / description	VARCHAR/TEXT	
assignments.due_at	TIMESTAMPTZ	
assignments.points_possible	NUMERIC(6,2)	
assignments.submission_types	TEXT[]	e.g., ["file","text"].
assignments.rubric_id	UUID FK -> rubrics	Optional.
submissions.id	UUID PK	
submissions.assignment_id	UUID FK -> assignments	
submissions.user_id	UUID FK -> users	
submissions.content	JSONB	text file_url external_url.
submissions.submitted_at	TIMESTAMPTZ	
submissions.is_late	BOOLEAN	
submissions.grade	NUMERIC(6,2)	NULL until graded.
submissions.feedback	TEXT	Instructor comment.

submissions.graded_at	TIMESTAMPTZ	
-----------------------	-------------	--

Relationships: Submissions belong to assignments and users. Grade written to gradebook.

quizzes & questions

Purpose: Quiz definitions and question bank.

Field	Type	Notes
quizzes.id	UUID PK	
quizzes.course_id	UUID FK -> courses	
quizzes.title	VARCHAR(255)	
quizzes.time_limit_seconds	INTEGER	NULL = no limit.
quizzes.attempt_limit	INTEGER	NULL = unlimited.
quizzes.shuffle_questions	BOOLEAN	
quizzes.available_from / until	TIMESTAMPTZ	
questions.id	UUID PK	
questions.bank_id	UUID FK -> question_banks	
questions.type	ENUM(mc,ms,tf,short,numeric,essay,matching)	
questions.content	JSONB	Text, media attachments.
questions.correct_answer	JSONB	Structured per type.
questions.points	NUMERIC(5,2)	
quiz_questions	Junction: quiz_id + question_id + position	
quiz_attempts	user_id + quiz_id + status + score + started_at + submitted_at	Per-attempt record.
quiz_responses	attempt_id + question_id + response + is_correct + score	Per-question answer.

Relationships: Questions belong to a question_bank. Quizzes reference questions via quiz_questions.

grades

Purpose: Unified gradebook entries.

Field	Type	Notes
id	UUID PK	
course_id	UUID FK -> courses	
user_id	UUID FK -> users	
grade_item_type	ENUM(assignment,quiz,simulation,discussion)	Polymorphic source.
grade_item_id	UUID	ID in respective table.
score	NUMERIC(6,2)	Actual score earned.
points_possible	NUMERIC(6,2)	Max score.
is_override	BOOLEAN	True if manually overridden.

override_note	TEXT	
graded_by	UUID FK -> users	NULL for auto-graded.
graded_at	TIMESTAMPTZ	

Relationships: Aggregated for course total grade calculation. One record per (user, grade_item).

enrollments

Purpose: Student-course enrollment records.

Field	Type	Notes
id	UUID PK	
user_id	UUID FK -> users	
course_id	UUID FK -> courses	
status	ENUM(active,pending,dropped,completed)	
enrolled_at	TIMESTAMPTZ	
completed_at	TIMESTAMPTZ	
final_grade	NUMERIC(5,2)	Calculated on course completion.

Relationships: UNIQUE(user_id, course_id). Has one progress record per enrollment.

certificates

Purpose: Issued completion certificates.

Field	Type	Notes
id	UUID PK	
user_id	UUID FK -> users	
course_id	UUID FK -> courses	
template_id	UUID FK -> certificate_templates	
issued_at	TIMESTAMPTZ	
verification_token	VARCHAR(64) UNIQUE	Public verification hash.
pdf_url	TEXT	Generated PDF stored in S3.

Relationships: Belongs to user and course. Verification_token used in public verify URL.

announcements, discussions, messages

Purpose: Communication entities.

Field	Type	Notes
announcements: id, course_id, author_id, title, body, is_pinned, published_at		

discussion_topics: id, course_id, author_id, title, body, is_graded, due_at		
discussion_posts: id, topic_id, author_id, parent_id (for replies), body, created_at		
messages: id, sender_id, recipient_id, subject, body, read_at, created_at		
notifications: id, user_id, type, title, body, link, read_at, created_at		

Relationships: Announcements and discussion_topics scoped to a course. Messages are direct user-to-user.

audit_logs

Purpose: Immutable audit trail.

Field	Type	Notes
id	UUID PK	
actor_id	UUID FK -> users	Who performed the action.
action	VARCHAR(100)	e.g., "user.create", "grade.override".
entity_type	VARCHAR(100)	e.g., "User", "Course".
entity_id	UUID	ID of affected entity.
delta	JSONB	Before/after snapshot of changed fields.
ip_address	INET	
created_at	TIMESTAMPTZ	

Relationships: Write-only table; no update/delete allowed at application level.

10. API Requirements

All endpoints are prefixed with `/api/v1`. Authentication via Bearer JWT unless marked [Public]. Responses use JSON. Errors follow RFC 7807 Problem Details format.

10.1 Auth

Method	Endpoint	Description	Auth Role
POST	<code>/auth/register</code>	Register a new user account.	Public
POST	<code>/auth/login</code>	Authenticate and receive JWT.	Public
POST	<code>/auth/logout</code>	Invalidate current session.	Any
POST	<code>/auth/refresh</code>	Refresh JWT using refresh token.	Any
POST	<code>/auth/forgot-password</code>	Send password reset email.	Public
POST	<code>/auth/reset-password</code>	Set new password via reset token.	Public
POST	<code>/auth/sso/{provider}</code>	Initiate OAuth/SAML SSO flow.	Public
GET	<code>/auth/sessions</code>	List active sessions for current user.	Any
DELETE	<code>/auth/sessions/{id}</code>	Revoke a specific session.	Any

10.2 Users

Method	Endpoint	Description	Auth Role
GET	<code>/users</code>	List users (paginated, filterable by role/department).	Admin
POST	<code>/users</code>	Create a single user.	Admin
POST	<code>/users/bulk-import</code>	Upload CSV for bulk user creation.	Admin
GET	<code>/users/{id}</code>	Get user profile.	Admin, Self
PATCH	<code>/users/{id}</code>	Update user profile or status.	Admin, Self
DELETE	<code>/users/{id}</code>	Soft-delete (anonymize) a user.	Admin
GET	<code>/users/me</code>	Current user's profile and context.	Any
PATCH	<code>/users/me/avatar</code>	Upload profile avatar.	Any

10.3 Courses

Method	Endpoint	Description	Auth Role
GET	<code>/courses</code>	List courses (catalog — published only for students; all for instructors).	Any
POST	<code>/courses</code>	Create a new course.	Instructor
GET	<code>/courses/{id}</code>	Get course details.	Any (enrolled or public)
PATCH	<code>/courses/{id}</code>	Update course metadata.	Instructor (owner)
POST	<code>/courses/{id}/publish</code>	Publish a draft course.	Instructor (owner)

POST	/courses/{id}/archive	Archive a course.	Instructor, Admin
POST	/courses/{id}/clone	Clone a course.	Instructor
GET	/courses/{id}/enrollments	List enrollments for a course.	Instructor, Admin
POST	/courses/{id}/enrollments	Enroll user(s) in a course.	Instructor, Admin, Student (self)
DELETE	/courses/{id}/enrollments/{userId}	Drop or remove enrollment.	Instructor, Admin, Student (self)

10.4 Modules & Lessons

Method	Endpoint	Description	Auth Role
GET	/courses/{courseId}/modules	List modules with lessons.	Enrolled user
POST	/courses/{courseId}/modules	Add a module.	Instructor
PATCH	/courses/{courseId}/modules/{id}	Update module title, position, unlock settings.	Instructor
DELETE	/courses/{courseId}/modules/{id}	Remove a module.	Instructor
POST	/courses/{courseId}/modules/{id}/lessons	Add a lesson.	Instructor
PATCH	/courses/{courseId}/modules/{id}/lessons/{lessonId}	Update lesson content.	Instructor
POST	/courses/{courseId}/modules/{id}/lessons/{lessonId}/complete	Mark lesson complete.	Student

10.5 Simulations

Method	Endpoint	Description	Auth Role
GET	/simulations	Browse simulation catalog.	Any
POST	/simulations	Register a new simulation.	Simulation Developer
GET	/simulations/{id}	Get simulation metadata.	Any
PATCH	/simulations/{id}	Update simulation metadata/config.	Simulation Developer, Admin
POST	/simulation-sessions	Start a new simulation session.	Student
GET	/simulation-sessions/{id}	Get session state.	Student (own), Instructor
POST	/simulation-sessions/{id}/events	Ingest step events.	Simulation Engine (token-auth)
POST	/simulation-sessions/{id}/complete	Submit completion payload.	Simulation Engine
GET	/courses/{courseId}/simulation-sessions	List all sessions for a course simulation.	Instructor

10.6 Assignments & Quizzes

Method	Endpoint	Description	Auth Role
POST	/courses/{courseId}/assignments	Create assignment.	Instructor
GET	/assignments/{id}/submissions	List submissions.	Instructor
POST	/assignments/{id}/submissions	Submit assignment.	Student
PATCH	/submissions/{id}/grade	Grade a submission.	Instructor, TA
POST	/courses/{courseId}/quizzes	Create quiz.	Instructor
POST	/quizzes/{id}/attempts	Start a quiz attempt.	Student
POST	/quiz-attempts/{id}/submit	Submit completed quiz.	Student
GET	/quizzes/{id}/analytics	Per-question statistics.	Instructor

10.7 Grades & Analytics

Method	Endpoint	Description	Auth Role
GET	/courses/{courseId}/gradebook	Full gradebook grid.	Instructor, TA
GET	/courses/{courseId}/grades/me	Student's own grades.	Student
PATCH	/grades/{id}/override	Override a grade.	Instructor
GET	/courses/{courseId}/gradebook/export	Export CSV.	Instructor, Admin
GET	/analytics/institutions/{id}/overview	Institution KPI dashboard.	Admin
GET	/analytics/courses/{id}/progress	Course progress report.	Instructor
GET	/analytics/simulations/{id}/performance	Simulation analytics.	Instructor, Simulation Developer
GET	/analytics/students/{id}/progress	Individual student analytics.	Instructor, Student (self)

10.8 Notifications

Method	Endpoint	Description	Auth Role
GET	/notifications	List notifications for current user.	Any
PATCH	/notifications/{id}/read	Mark notification as read.	Any
POST	/notifications/mark-all-read	Mark all notifications read.	Any
GET	/messages	List message threads.	Any
POST	/messages	Send a direct message.	Any
GET	/messages/{threadId}	Get message thread.	Participants only

11. UI Pages and Dashboards

Page	Route	Purpose	Key Components	Access
Landing Page	/	Public	Showcase platform features, sample simulations, pricing, testimonials, and institution login. CTA: "Start Free Trial" / "Institutional Login".	Guest
Login / Register	/login, /register	Auth flow	Email/password and SSO login; new account registration; email verification prompt.	Guest
Student Dashboard	/dashboard	Post-login hub	My Courses progress cards, recent activity feed, upcoming deadlines, notifications, quick-launch recent simulation.	Student
Instructor Dashboard	/instructor	Teaching hub	Course management cards, submission inbox, grade queue, recent student activity, analytics summary.	Instructor, TA
Admin Dashboard	/admin	Administration hub	Platform KPIs, user management quick links, recent audit log events, system health monitors.	Admin, Super Admin
Course Catalog	/courses	Browsing	Grid/list of published courses; filters by domain, level, type, instructor; keyword search.	All
Course Details	/courses/:id	Pre-enrollment	Syllabus, instructor info, module list preview, enrollment button, reviews, simulation count.	All
Course Homepage	/courses/:id/home	Enrolled learner hub	Announcements, progress bar, next lesson shortcut, calendar events, module navigation sidebar.	Enrolled users
Lesson / Content Page	/courses/:id/lessons/:lessonId	Content delivery	Rich text / video / file / SCORM viewer; lesson navigation; progress toggle; discussion link.	Enrolled users

Simulation Catalog	/simulations	Lab browsing	Simulation cards with domain filter, level, duration, rating; search; Featured/New badges.	All
Simulation Player	/simulations/:id/play	Simulation execution	Fullscreen simulation iframe; instructions sidebar; progress indicator; hint button; save/exit controls.	Student
Assignment Page	/courses/:id/assignments/:asgld	Submission	Assignment brief; file upload / rich text entry; submission history; rubric preview.	Student, Instructor
Quiz Page	/courses/:id/quizzes/:qzld	Assessment	Question renderer; timer; navigation between questions; auto-save indicator; submit button.	Student
Gradebook	/courses/:id/gradebook	Grade management	Student-by-item grid; column filters; grade entry inline; export button; statistics row.	Instructor, TA, Student (own)
Calendar	/calendar	Scheduling	Month/week/day views; course events color-coded; assignment deadlines; office hours; iCal export.	All
Messages	/messages	Communication	Inbox thread list; conversation view; compose modal; file attachment; search.	All
Reports & Analytics	/analytics	Insights	Report selector; date range picker; chart visualizations; data table; export controls.	Instructor, Admin
Settings	/settings	Configuration	Profile settings, notification preferences, security (password, MFA, sessions), institution settings (admin).	All
Question Bank	/question-bank	Authoring	Question list with filters; create/edit question form; bulk import; tag management.	Instructor, Content Creator
Simulation Authoring	/simulations/new	/simulations/:id/edit	Simulation metadata form; SCORM upload or LTI config; step definition; competency mapping; test launch.	Simulation Developer

12. Role-Based Permissions Matrix

Legend: ✓ = Full access, L = Limited (own items only or read-only), — = No access, A = Approval required.

Resource / Action	Super Admin	Inst. Admin	Dept. Mgr	Instructor	TA	Student	Content Creator	Sim. Developer	Guest
Manage Institutions	✓	—	—	—	—	—	—	—	—
Manage Users (all)	✓	✓	—	—	—	—	—	—	—
Manage Users (dept)	✓	✓	✓	—	—	—	—	—	—
View User List	✓	✓	✓	L	—	—	—	—	—
Create/Edit Courses	✓	✓	✓	✓	—	—	—	—	—
Publish Courses	✓	✓	✓	✓	—	—	—	—	—
Delete Courses	✓	✓	—	L	—	—	—	—	—
Create Modules/Lessons	✓	✓	—	✓	—	—	✓	—	—
Upload Content	✓	✓	—	✓	—	—	✓	—	—
Create Assignments	✓	✓	—	✓	—	—	—	—	—
Grade Submissions	✓	✓	—	✓	✓	—	—	—	—
Create Quizzes	✓	✓	—	✓	—	—	✓	—	—
View All Grades	✓	✓	✓	✓	L	—	—	—	—
View Own Grades	✓	✓	✓	✓	✓	✓	—	—	—
Override Grades	✓	✓	—	✓	—	—	—	—	—
Export Gradebook	✓	✓	✓	✓	—	—	—	—	—
Create Simulations	✓	—	—	—	—	—	—	✓	—
Add Sim to Course	✓	✓	—	✓	—	—	—	✓	—
Launch Simulations	✓	✓	✓	✓	✓	✓	L	✓	L (demo)
View Sim Analytics	✓	✓	✓	✓	—	L	—	✓	—
Post Announcements	✓	✓	✓	✓	✓	—	—	—	—
Post in Discussions	✓	✓	✓	✓	✓	✓	—	—	—

Moderate Discussions	✓	✓	✓	✓	✓	—	—	—	—
Send Messages	✓	✓	✓	✓	✓	✓	✓	✓	—
View Analytics Dashboards	✓	✓	✓	✓	—	L	—	L	—
Export Reports	✓	✓	✓	✓	—	—	—	—	—
Manage Roles	✓	✓	—	—	—	—	—	—	—
View Audit Logs	✓	✓	—	—	—	—	—	—	—
System Settings	✓	L	—	—	—	—	—	—	—
Issue Certificates	✓ (auto)	✓	—	A	—	—	—	—	—

13. Simulation Learning Features

13.1 Simulation Metadata Schema

Field	Description
title	Human-readable simulation name displayed in catalog and course.
subject_domain	Subject area (e.g., Thermodynamics, Cell Biology).
learning_objectives	List of competency-aligned outcome statements.
prerequisites	Simulations or competencies required before launch.
difficulty_level	Introductory, Intermediate, Advanced.
estimated_duration	Typical completion time in minutes.
language_support	Array of ISO 639-1 language codes.
launch_type	SCORM LTI Internal.
scoring_config	JSON: step weights, hint_penalty_pct, attempt_bonus.
step_definitions	Ordered array: { step_id, name, max_score, competency_ids[] }.
tags	Free-form keywords for catalog search.

13.2 Experiment Step Flow

Each simulation is structured as a sequence of experiment steps. The platform tracks progress at the step level:

- Step Entry Event: logged when student navigates to a step.
- Interaction Events: tool selections, data entries, and parameter adjustments logged with value and timestamp.
- Step Completion Event: logged when student meets the step success criteria (correct answer, correct procedure, or time spent).
- Hint Requested Event: logged with hint level (1, 2, or 3); each level reduces step score by the configured penalty.

13.3 Auto-Grading Logic

Component	Formula / Rule
Step Score	$\text{step_score} = \text{step_max_score} - (\text{hints_used} \times \text{hint_penalty_per_hint})$

Session Score	$\text{session_score} = \text{SUM}(\text{step_scores}) / \text{SUM}(\text{step_max_scores}) \times \text{simulation_max_score}$
Attempt Modifier	If multiple attempts enabled: best_of_N, average_of_N, or last_attempt (configurable per simulation)
Grade Entry	Final session score written to grades table; scaled to assignment point value if set differently from simulation max_score.
Partial Completion	If student exits without completing: $\text{partial score} = \text{steps_completed_score} / \text{total_max_score} \times \text{simulation_max_score}$

13.4 Competency Proficiency Tracking

Each simulation step maps to one or more learning competencies. The platform maintains a running proficiency model per student:

- Evidence Weight: each step completion contributes evidence to mapped competencies. Evidence weight = $\text{step_score_pct} \times \text{step_weight}$.
- Proficiency Levels: Not Yet (<40%), Developing (40–69%), Proficient (70–89%), Mastered ($\geq 90\%$).
- Decay: proficiency decays if a student has not engaged with evidence-generating activities for >60 days (configurable).
- Cross-Simulation Aggregation: competency proficiency aggregated across all simulations and assignments that contribute evidence.

13.5 Feedback Engine

Feedback Type	Description
Step-Level Feedback	Immediate after step: correct/incorrect indicator + explanation text configured by simulation developer.
Hint-Level Feedback	Progressive disclosure: Hint 1 = conceptual reminder, Hint 2 = worked example, Hint 3 = direct answer.
End-of-Simulation Report	Summary: score per step, total score, time taken, competencies addressed, weakest areas.
Instructor Feedback View	Instructor can see per-student step-level event log for detailed intervention.
Adaptive Suggestions	System recommends review lessons or remedial simulations based on lowest-scoring competencies (future feature).

14. Reports and Analytics

Report Name	Description	Primary Users	Key Metrics	Export
Student Progress Report	Individual student's completion, grades, simulation performance, and time-on-platform within a course or all courses.	Instructor, Admin	% Complete, avg grade, simulation scores, last activity date.	CSV, PDF
Course Completion Report	Aggregate completion funnel and grade distribution for a course.	Instructor, Dept Mgr, Admin	Enrollment count, completion rate, avg grade, dropout rate, completion by module.	CSV, PDF
Simulation Performance Report	Per-simulation analytics including step-level failure patterns.	Instructor, Sim Developer	Avg score, avg duration, hint usage %, most-failed step, score distribution.	CSV
Quiz Results Report	Per-quiz item analysis with question difficulty and distractor performance.	Instructor	Avg score, per-question correct%, difficulty index, discrimination index.	CSV, PDF
Assignment Submission Report	Submission rates, late rates, and grade distribution per assignment.	Instructor	Submission rate, on-time%, avg grade, late submission%, pending grades.	CSV
Instructor Activity Report	Instructor engagement with their courses (logins, content updates, grading turnaround).	Institution Admin	Days logged in, courses updated, avg grading time, announcements posted.	CSV
Institution Analytics Dashboard	Aggregated KPIs for the institution across all courses and domains.	Institution Admin, Super Admin	MAU, DAU, courses active, simulation hours logged, outcome achievement%.	Dashboard, CSV
Domain-Level Analytics	Comparative analytics across all courses within a domain.	Dept Manager, Admin	Enrollment trends, course completion rates, top/bottom performing courses.	CSV

Engagement Report	User engagement patterns: session frequency, feature usage, drop-off points.	Admin	DAU/MAU ratio, avg session length, top features used, inactivity segments.	CSV
Competency Achievement Report	Aggregate competency proficiency distribution across student population.	Instructor, Dept Mgr, Admin	% students at each proficiency level per competency, trend over time.	CSV, PDF

15. Development Roadmap and Deliverables

15.1 Recommended MVP Features (Phase 1 — 6 months)

The Minimum Viable Product should deliver a functional LMS with core simulation capabilities:

Feature Area	Scope
Authentication & User Management	User registration, email verification, login/logout, password reset, basic RBAC (Student, Instructor, Admin).
Institution Setup	Single-institution deployment, department creation, user bulk import.
Course Management	Course CRUD, module/lesson creation, rich-text + file lessons, course publishing.
Enrollment	Student self-enrollment, instructor-managed enrollment, course catalog.
Simulations (Basic)	SCORM 2004 import, simulation player with instructions panel, session tracking, score writing to gradebook.
Assignments	File and text submission, instructor grading, basic feedback, late submission flag.
Quizzes	Multiple choice and true/false questions, timed quiz, auto-grading, attempt limit.
Gradebook	Per-course grade grid, grade entry, student grade view, CSV export.
Notifications	Email notifications for key events, in-app notification center.
Student Dashboard	My Courses, progress bars, upcoming deadlines, recent grades.
Instructor Dashboard	Course management, submission inbox, grade queue.
Admin Dashboard	User management, course oversight, basic analytics.

15.2 Phase 2 — Months 7–12 (Advanced Features)

Feature Area	Enhancements
Multi-tenant	Multiple institutions, white-labeling, SSO per institution (SAML, Google, Microsoft).
LTI 1.3	Full LTI 1.3 Tool Consumer and Provider; external simulation catalog integration.
Simulation Advanced	LTI-integrated simulations, custom scoring config, step-level event analytics, competency mapping.
Rubrics	Rubric builder and attachment to assignments; criterion-level feedback.
Question Bank	Shared question bank, question tagging, quiz question pooling and randomization.
Discussions & Messaging	Threaded course discussions, graded discussions, direct messaging.
Calendar & Announcements	Full calendar view, pinned announcements, iCal feed export.
Certificates	Certificate template builder, auto-issue on course completion, public verification URL.

Analytics	Course completion reports, simulation performance reports, student progress reports.
MFA	TOTP multi-factor authentication, admin-enforced MFA for roles.

15.3 Phase 3 — Months 13–18 (Scale & Intelligence)

Feature Area	Features
Competency Engine	Full competency proficiency model, decay, cross-activity aggregation, visual dashboard.
Adaptive Suggestions	AI-driven remediation recommendations based on competency gaps.
Advanced Analytics	Engagement reports, domain-level dashboards, ClickHouse event analytics, instructor activity reports.
Payment / Subscriptions	Stripe-based institution subscription billing, per-course payment for open learners.
Mobile PWA	Progressive Web App optimization for mobile, offline lesson access, background sync.
Accessibility Audit	Full WCAG 2.1 AA audit and remediation across all pages and simulations.
Localization	Arabic (RTL), French, Spanish, German UI localization.
API Marketplace	Public API documentation portal, webhook support, third-party developer onboarding.
xAPI / LRS	xAPI statement generation for all tracked activities; Learning Record Store integration.
AI-Assisted Authoring	AI content suggestions for lesson text, quiz question generation, simulation feedback text.

15.4 Final Deliverables Summary

Deliverable	Description
This SRS Document	Complete software requirements specification including all sections in this document.
Use Case List	15 detailed use cases covering all primary actor-system interactions.
User Sequence Flows	8 sequence flows covering key platform workflows from actor through to database/services.
Database Schema	Entity definitions for 20+ tables with fields, types, and relationships.
API Endpoint List	80+ REST API endpoints organized by module with method, path, description, and auth requirements.
UI Page Inventory	20 page definitions with purpose, key components, and access control.
Permissions Matrix	9-role × 30-resource RBAC matrix covering all major platform operations.
Development Roadmap	3-phase roadmap with MVP scope, phase 2 enhancements, and phase 3 scaling features.
Architecture Proposal	Layer-by-layer system architecture with technology recommendations.

Simulation Feature Spec	Detailed specification for simulation metadata, scoring, competency tracking, and feedback engine.
-------------------------	--

Document Control
Version: 1.0 | Status: Draft | Prepared for: SimuLearn Product & Engineering Team | Review Cycle: Quarterly | Next Review: September 2026